

Self Study: Getting Started in Python

Raul Hochuli (raul.hochuli@unibas.ch)

Learning Goals (self study as home work)

- Get a very basic understanding of the most important technical terms around "Python Programming".
- Open a Python file and execute its content.
- Complete the missing parts in some example Python code
- *I assume you have a Google Account or are able to set one up yourself. If you are not familiar with Python at all, this handout should help you with the first steps on your own. If you still struggle after going through this handout, please come and see me / email me, so we can close these gaps.*

The Ins and Outs about Python

When we say "Python", we don't mean a single program or app, but a number of technical components that together run the Python language. While you don't need to know these details by heart for our course, but having heard of them will save you many problems, should you ever choose to use Python again (let's say for an upcoming bachelor's or master's thesis).

Think of Python as a (programming) language - a way for humans to give instructions to a computer, just like using words and grammar to communicate with another person. When you write Python, you're telling the computer what to do and how to do it.

To write and run Python code, you usually use a Console or an IDE¹. Think of this like pen and paper (or today your Word document or notes app): it's the place where you write your ideas down and try them out. The IDE lets you

¹Integrated Development Environment

run commands and helps you organize your code into bigger programs. Popular examples for IDEs are e.g. Google Colab, Jupyter PyCharm, Spyder, VS Code.

For Python to run, you need a Python interpreter and package distributor. You can imagine the interpreter to be the very basic dictionary and packages to be more sophisticated, topic-related textbooks that can help you write your thoughts in a topic-specific manner. Probably the most popular distributor (already including the interpreter) is Anaconda, popular packages are e.g. pandas or numpy.

Last but not least, you have different file types, straight Python files (".py") and cell-based Jupyter Notebooks (".ipynb"). For us, this distinction is really minimal; think of it as different paper types, e.g. paper with preprinted lines or squares on it.

First Commands + Opening a File

In our course, we want to minimize technicalities as much as possible and focus more on content about resource economics. This is why we use [Google Colab](#), a simple Google extension like Gmail, Google Maps, which operates on your GDrive and brings with it already all the technical requirements mentioned above. For teaching, I will use Jupyter Notebooks, as they are most suitable for coding and written explanations at the same time. This video shows you [how to get to Google Colab and how to open a file](#).

Now, try to download the first Jupyter Notebooks from ADAM, open them in Google Colab and to follow the instructions mentioned in them.